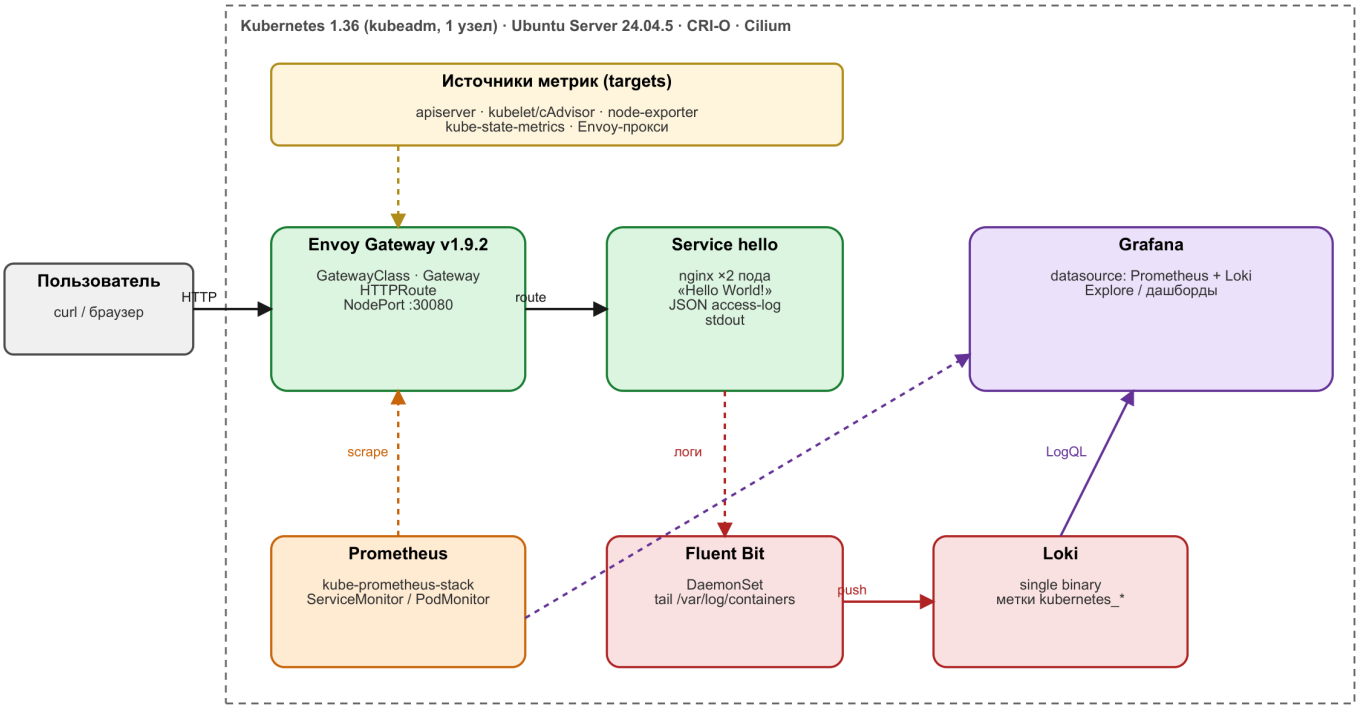
Паспорт решения: Kubernetes + Gateway API + Prometheus + Fluent Bit/Loki

Репозиторий: https://github.com/artemid00/mts-engineer-hack-devops · ветка main · Ubuntu Server 24.04.5 LTS · полная инструкция — README.md

1. Архитектура и состав решения

Параметр	Значение
Версия Kubernetes	1.36 (один узел: control-plane + нагрузка)
Способ развёртывания Kubernetes	kubeadm (официальные пакеты kubernetes.io), CRI-O v1.36, CNI Cilium 1.20.2
Реализация Gateway API	Envoy Gateway v1.9.2 (Gateway API CRD v1.6.1): GatewayClass, Gateway, HTTPRoute, EnvoyProxy
Инструменты автоматизации	Ansible (роли), Helm (все компоненты и собственный чарт), GNU Make, shell
Логирование	**Fluent Bit** (chart 0.58.2) → Loki 3.7.8 (chart 18.13.7), просмотр в Grafana
Развёртывание Prometheus	Helm: kube-prometheus-stack 91.8.2 (Prometheus Operator, Grafana, node-exporter, kube-state-metrics)
Тестовая ОС	Ubuntu Server 24.04.5 LTS (чистая VM: 4 vCPU, 8 ГБ RAM)
Приложение	nginx-unprivileged 1.27 (публичный образ), ответ «Hello World!», JSON access-логи в stdout

Схема: пользователь → Gateway API (Envoy) → Service → nginx; Prometheus собирает метрики кластера и Envoy; Fluent Bit читает логи контейнеров и отправляет в Loki; Grafana показывает метрики и логи.



Запуск: `sudo -E make deps cluster deploy`, проверка: `sudo make verify` (повтор безопасен).

2. Реализованный функционал

Обязательные пункты

Пункт	Как реализовано и почему	Как проверить эксперту
Kubernetes	Роли Ansible: подготовка ОС (swap, модули, sysctl), CRI-O, kubelet/kubeadm/kubectl, `kubeadm init` с снятием taint, Cilium через Helm. kubeadm выбран как приоритетное вендор-нейтральное решение; одноузловой вариант — минимум ресурсов.	`kubectl get nodes` → Ready; `kubectl get pods -A` — все Running.
Веб-приложение	nginx-unprivileged, 2 реплики, ConfigMap с конфигом: «Hello World!» на /, /healthz для проб, access-log в JSON в stdout, error-log в stderr — их собирает Fluent Bit.	`kubectl -n demo get deploy,pods`; `curl http://<IP>:30080/` → Hello World!
Gateway API	Envoy Gateway ставится Helm-чартом (вместе с CRD). GatewayClass `envoy` с параметрами EnvoyProxy (сервис NodePort 30080), Gateway `demo-gw` :80, HTTPRoute `hello` → Service. Реализация открытая, без привязки к облаку.	`kubectl -n demo get gateway,httproute` (PROGRAMMED=True); `curl http://<IP>:30080/`
Мониторинг	kube-prometheus-stack: Prometheus собирает apiserver, kubelet/cAdvisor, node-exporter, kube-state-metrics, Envoy. Оператор + ServiceMonitor/ PodMonitor — стандартный и расширяемый способ.	`kubectl -n monitoring port-forward svc/kps-kube-prometheus-stack-prometheus 9090` → /targets (UP); запрос `up{job="apiserver"}` = 1.
Логирование	Fluent Bit DaemonSet читает `/var/log/containers`, обогащает метаданными Kubernetes (фильтр kubernetes, Buffer_Size 256k) и отправляет в Loki. Loki — лёгкое хранилище, индексирует только метки; логи доступны в Grafana.	После `curl` к приложению: `{kubernetes_namespace_name="demo"}` в Grafana Explore или запрос к API Loki (README, раздел 3).
Ubuntu 24.04	Весь стенд развёрнут и проверен на Ubuntu Server 24.04.5 LTS.	`lsb_release -a`
Автоматизация	Два плейбука (`cluster.yml`, `addons.yml`), Makefile, версии зафиксированы в `group_vars/all.yml`; `helm upgrade --install` и проверка `admin.conf` дают идемпотентность; `make clean` возвращает узел в исходное состояние.	`sudo -E make cluster deploy` дважды подряд; `sudo make verify` завершается OK.
Документация	README.md: архитектура, версии, требования, команды, три проверки, ограничения.	README.md в корне репозитория.

Дополнительные улучшения

Что	Как и зачем	Как проверить
Собственный Helm-чарт `charts/demo`	Приложение, Gateway, HTTPRoute, EnvoyProxy, PodMonitor и datasource в одном чарте с параметрами (реплики, образ, NodePort, hostname) — ресурсы не правятся вручную.	`helm list -n demo`; `helm template charts/demo`
Метрики Envoy (HTTP)	PodMonitor на Envoy-прокси: число запросов, коды ответов, latency на уровне Gateway.	Prometheus → Targets (envoy); запросы `envoy_http_downstream_rq_xx`.
Grafana с Prometheus и Loki	Datasource Loki создаётся автоматически (ConfigMap с меткой sidecar), единый интерфейс для метрик и логов.	Grafana → Connections → Data sources.
Безопасность и надёжность	nginx не от root, `readOnlyRootFilesystem`, `drop ALL`, seccomp RuntimeDefault, probes, лимиты ресурсов, 2 реплики; checksum-аннотация перекатывает поды при смене конфига; секреты — только через переменные окружения.	`kubectl -n demo get deploy hello -o yaml`
Самолечение установки и проверка	Роль Cilium ждёт CoreDNS и при зависании перезапускает агент CNI; `verify-all.sh` проверяет Gateway, Prometheus и логи в Loki одной командой; `reset.sh` — полный сброс.	`sudo make verify`, `sudo make clean`

3. Ревью работы и потенциальное масштабирование

Главная особенность решения

Полная воспроизводимость на чистой Ubuntu 24.04 одной последовательностью команд: кластер на kubeadm, Gateway API, мониторинг и логи ставятся идемпотентно, версии зафиксированы, а `make verify` автоматически подтверждает работу трёх обязательных частей. Все компоненты развёртываются Helm-чартами — без ручного редактирования ресурсов.

Самое сложное решение

Выбор CNI и реализации Gateway API: у Cilium есть собственная реализация Gateway API, но я оставил Cilium только CNI, а Gateway API отдал отдельному Envoy Gateway — чёткое разделение ответственности и независимость от CNI. Второй выбор — Loki вместо Elasticsearch: на одном узле он требует в разы меньше ресурсов, а для задачи достаточно поиска по меткам.

Предложения по развитию

- **TLS и несколько маршрутов:** cert-manager (бесплатно), HTTPS-листенер, маршрутизация по hostname/path, canary-распределение трафика (traffic splitting) — требуется только DNS-имя.
- **Высокая доступность:** 3 control-plane + HAProxy/keepalived, несколько worker-узлов, внешний LoadBalancer (MetalLB или Cilium L2) — нужны 3+ VM и свободные IP.
- **Постоянное хранение:** Longhorn/local-path для Prometheus и Loki, хранение чанков Loki в S3-совместимом хранилище (MinIO).
- **Алертинг:** Alertmanager с правилами SLA (доступность, p95 latency, 5xx) и уведомлениями в Telegram/почту.
- **CI/CD:** pipeline (линтеры ansible-lint/yamllint, `helm lint`, kubeconform, smoke-тест в kind) и GitOps (Argo CD).
- **Телеком-специфика:** Gateway API как точка входа для сервисов с SLA (API, биллинг, абонентские порталы) с rate limiting и mTLS на Envoy; хранение логов по требованиям ФЗ-374 (срок и сохранность); мультиарендность по namespace и NetworkPolicy; распределённый кластер по площадкам (Multi-cluster, Cilium ClusterMesh) для отказоустойчивости; трассировка (OpenTelemetry/Tempo) для разбора инцидентов.